

KubeQuest — Sécurité : comment nous avons trouvé et corrigé les vulnérabilités (support de présentation)

Périmètre : l'application PHP (Laravel) `app-master/` + sa chaîne de déploiement (Dockerfile, docker-compose, manifestes Helm/K8s) + le durcissement du cluster live.

1. Notre méthode : comment nous avons TROUVÉ les vulnérabilités

Nous n'avons pas fait une simple relecture « à l'œil ». Nous avons appliqué une **démarche d'audit structurée en 3 temps** :

Temps 1 — Audit multi-axes (7 angles d'attaque en parallèle)

Nous avons passé le code au crible selon **7 lentilles** distinctes, chacune cherchant un type de faille précis : 1. **Secrets** — identifiants/clés en clair dans le dépôt. 2. **Divulgation / debug** — mode debug, messages d'erreur exposant des infos. 3. **Injection / flux de données** — SQL injection, mass-assignment, désérialisation. 4. **Contrôle d'accès / CSRF / CORS** — endpoints non authentifiés, politiques permissives. 5. **XSS** — encodage des sorties HTML. 6. **Durcissement conteneur** — Dockerfile (root, deps de dev, image de base). 7. **CVE des dépendances** — versions vulnérables de librairies.

Temps 2 — Vérification contradictoire (on essaie de RÉFUTER chaque trouvaille)

Chaque constat candidat est **réexaminé par un second analyste dont le but est de le démolir** : est-ce réellement exploitable dans CE contexte ? La sévérité est-elle juste ? Est-ce un faux positif déjà mitigé par un défaut de Laravel ? Ce filtre évite les « fausses alertes » qui décrédibilisent un rapport.

Temps 3 — Vérification en ligne des CVE (sources officielles)

Pour chaque CVE cité, nous sommes allés **vérifier sur NVD (NIST) et les GitHub Security Advisories** : versions affectées, version corrigée, score CVSS. **Aucun numéro de CVE inventé.**

Bilan chiffré

37 constats candidats → **23 confirmés** → **18 problèmes distincts** (après dédoublonnage), **14 pistes écartées** (faux positifs ou déjà corrigés).

Sévérité (après vérification)	Nombre
High	1
Medium	6
Low	9
Info	2

Constat clé : l'application elle-même (un simple compteur, sans authentification ni entrée utilisateur) a une **surface d'attaque applicative très faible**. Les vrais risques sont l'**hygiène des secrets** (identifiants en clair) et l'**obsolescence** (framework en fin de vie).

2. Ce que nous avons TROUVÉ (les trouvailles marquantes)

● High — Identifiants de base de données (dont root MySQL) en clair

- Où : `app-master/docker-compose.yaml` lignes 32-33 (compte app) et 45-47 (compte MySQL + **mot de passe root**).
- Le problème : des mots de passe faibles (`app_password` , `app_root_password`) sont écrits en clair, **versionnés dans git**, et **copiés dans l'image Docker** (car `COPY` . sans `.dockerignore`).
- L'impact : quiconque lit le dépôt ou l'image obtient le compte applicatif ET le mot de passe root de MySQL.

● Medium — Clé applicative `APP_KEY` en clair (CWE-798)

- Une vraie clé de chiffrement Laravel (AES-256-CBC) committée dans `docker-compose.yaml` ligne 27.
- Nuance honnête (grâce à la vérification) : ramenée de « Critical » à « Medium » car l'app de prod utilise une autre clé et n'a ni authentification ni chiffrement à voler.

● Medium — CVE-2024-52301 (Laravel) — le seul CVE réellement applicable

- Vérifié sur NVD ([lien](#)) + [GHSA-gv7v-rgg6-548h](#), CVSS 8.7 (HIGH).
- Le problème : `laravel/framework` v8.83.27 est vulnérable (corrigé en 8.83.28). Quand la directive PHP `register_argc_argv` est activée (défaut de l'image), un attaquant peut **forcer l'environnement Laravel via la query string** de l'URL.

● Medium — Framework Laravel 8.x en fin de vie

- Plus de correctifs de sécurité depuis janvier 2023. **Vérifié** : même la dernière version 8.x (8.83.28) reste signalée par d'autres avis non corrigés (dont CVE-2025-27515). → Seule une **migration majeure** résout durablement.

● Medium — Secrets dans un ConfigMap au lieu d'un Secret (CWE-312)

- Le chart Helm écrivait `APP_KEY` / `DB_PASSWORD` / `REDIS` / `MAIL` dans un **ConfigMap** (lisible en clair, RBAC large) au lieu d'un **Secret**.

● Medium — Dépendances de dev embarquées en production

- Le Dockerfile fait `composer install` sans `--no-dev` → Ignition, Whoops, PHPUnit, Faker livrés dans l'image publique.

Les CVE que nous avons VÉRIFIÉS puis ÉCARTÉS (rigueur)

CVE	Verdict vérifié en ligne
CVE-2023-3824 / CVE-2023-3823 (PHP)	NON applicables : corrigés en PHP 8.2.8 , or notre image EST en 8.2.8. (Le premier audit s'était trompé sur la version corrigée — la vérification en ligne l'a rattrapé.)
CVE-2021-3129 (Ignition RCE)	Non exploitable : notre version 2.17.7 ≥ 2.5.2 (corrigée).
CVE-2018-15133 (RCE via APP_KEY)	Non applicable : corrigé depuis Laravel 5.6.30 (cookies JSON).

Cette diapo est un point fort : elle montre qu'on ne se contente pas de lister des CVE effrayants, on **vérifie s'ils s'appliquent vraiment**.

3. Comment nous avons CORRIGÉ

3.1 Correctifs dans le code / la configuration (commits, PR #5 & #6)

Vulnérabilité	Correctif appliqué
CVE-2024-52301	Ajout de <code>register_argc_argv = Off</code> dans l'image PHP (<code>Dockerfile</code> , dans <code>\$PHP_INI_DIR/conf.d</code>). Ça supprime la condition d'exploitation du CVE (l'attaque exige <code>register_argc_argv = On</code>). C'est la mitigation recommandée officiellement.
Identifiants en clair (docker-compose)	Les secrets (APP_KEY, DB_PASSWORD, mot de passe root) sont sortis du fichier et injectés via <code>\${VAR:?}</code> depuis un <code>.env non versionné</code> (échec immédiat si non défini = <i>fail-closed</i>). Fichier modèle <code>.env.docker.example</code> ajouté.
Fuite dans l'image	Ajout d'un <code>.dockerignore</code> (empêche <code>.env</code> , <code>docker-compose*</code> , <code>.git</code> d'entrer dans l'image ; <code>.gitignore</code> ne filtre PAS le build Docker).
Secrets en ConfigMap	Le chart Helm injecte désormais les secrets via <code>secretKeyRef</code> depuis un Secret Kubernetes (plus via le ConfigMap).

Pourquoi pas simplement passer à Laravel 8.83.28 ? Nous l'avons testé : l'outil d'audit de Composer **refuse** cette version car 8.83.28 reste affectée par d'autres avis (branche 8.x en fin de vie). La vraie solution est une **migration majeure**, donc nous avons choisi la **mitigation au niveau PHP** (sûre, immédiate) et documenté la migration comme suite.

3.2 Durcissement du cluster (déployé en LIVE et testé le jour du montage)

Au-delà du code, nous avons renforcé le cluster en exécution :

Mesure	Ce que ça apporte	Démo
OPA / Gatekeeper (validating webhook)	Refuse tout pod ne respectant pas la politique (ici : label obligatoire). Exigence sécurité du sujet.	<code>kubectrl run</code> sans label → refusé
NetworkPolicy	Isole l'app : seuls l'ingress et le monitoring peuvent l'atteindre ; les autres namespaces sont bloqués .	depuis <code>default</code> → timeout
cert-manager + HTTPS	Émission automatique d'un certificat TLS (auto-signé) sur l'ingress de l'app.	certificat <code>Ready=True</code>
SSO dex + oauth2-proxy	Authentification unifiée (OIDC) devant les outils. Exigence sécurité du sujet.	accès Prometheus non authentifié → 302 vers dex
Basic-auth Prometheus	Prometheus n'était pas protégé du tout.	login requis

⚠ Note importante à connaître (piège) : le dépôt contenait un manifeste OPA « maison » avec `failurePolicy: Fail` appliqué à **tout le cluster** et sans TLS fonctionnel. **Le déployer aurait bloqué la création de tout pod dans le cluster.** Nous l'avons **remplacé par Gatekeeper** (fail-open, exclut les namespaces système), qui est l'implémentation sûre et standard d'OPA.

3.3 Nettoyage & fiabilisation (état sain pour la démo)

- Ancien déploiement `laravel` en CrashLoop → mis à l'arrêt (l'app utilisée est `laravel-app-prod`).

- **Sauvegardes MySQL réparées** : il manquait une StorageClass → le PVC de backup restait bloqué ; ajout d'une StorageClass → **backups fonctionnels**.
 - Grafana/Prometheus exposés via ingress + **source de données Loki** ajoutée (logs visualisables).
-

4. Ce qui reste (transparence)

- **Migration Laravel** vers une version supportée (10/11/12) : la branche 8.x est EOL, aucune version « propre » n'existe.
 - **Redéploiement de l'image corrigée sur le cluster** : contrainte technique (image en `imagePullPolicy: Never` + pas de registre), le rebuild/redéploiement multi-nœuds est préparé mais non appliqué pour ne pas risquer la démo.
 - **SSO généralisé** à tous les outils (aujourd'hui : ArgoCD/Grafana ont leur propre login, Prometheus est derrière le SSO dex/oauth2-proxy).
-

5. Résumé express (à réciter en 45 secondes)

« Nous avons mené un audit de sécurité structuré : 7 axes d'analyse, une vérification contradictoire de chaque trouvaille, et une vérification en ligne de chaque CVE sur NVD. Résultat : 23 vulnérabilités confirmées, dont **un seul CVE réellement applicable, CVE-2024-52301**, que nous avons **neutralisé** en désactivant `register_argc_argv`. Nous avons aussi sorti tous les secrets du dépôt, corrigé la gestion des secrets Kubernetes, et durci le cluster live avec **OPA/Gatekeeper, des NetworkPolicies, HTTPS via cert-manager et un SSO dex/oauth2-proxy**. Point notable : nous avons vérifié que 3 CVE souvent cités **ne s'appliquaient pas** à notre version — la rigueur avant la peur. »